Machine Learning for Electronic Design Automation and Manufacturing Mock Exam – Solutions

Question 1

What loss function would you use in a classification task with 6 classes? Name additionally a loss function that you would not use and explain why you would not use it.

Solution:

For a classification task with 6 classes, a suitable loss function is

- **Categorical cross-entropy** (for one-hot encoded labels), or
- **Sparse categorical cross-entropy** (for integer class labels).

These losses are designed for multi-class classification problems. They compare the predicted class probabilities (typically obtained via a softmax output layer) with the true class labels and penalize incorrect predictions.

A loss function that we **would not** use is:

- **Mean Squared Error (MSE)**.

MSE measures squared differences between predicted values and targets and is more appropriate for regression. For multi-class classification, MSE does not model class probabilities well and leads to slower and less stable learning compared to cross-entropy-based losses. (Similarly, **binary cross-entropy** alone would be inappropriate because it is meant for binary or independent multi-label classification, not a single 6-class decision.)

Question 2

You have built the following neural network *model* for classification and saved 2 copies of it in the variables *model1* and *model2*.

```python
from keras.models import Sequential
from keras.layers import Dense, Conv2D, Flatten, BatchNormalization
from copy import deepcopy

input_dim = (28,28,3)
num_convolutional_filters = 32
output_dimension = 1

model = Sequential()
model.add(Conv2D(16, (3,3), activation='relu'))

model1 = deepcopy(model)
model2 = deepcopy(model)
```

(a) Write Python code to extend your *model1* to match the output dimension. Use at least one more convolutional layer with *num_convolutional_filters* filters and a fully connected layer. Don't forget to add the necessary hyperparameters (e.g. activation functions) with appropriate values.

(b) Do the same as point (a) with *model2* but this time with at least a *Batch-normalization* layer and a fully connected layer. Don't forget the necessary hyperparameters with appropriate values.
Hint: the Batchnormalization layer function does not need any parameter

(c) Suppose you are getting the data already prepared for your model in the variables *X_train*, *y_train*, *X_val*, *y_val*, *X_test*, *y_test*. Write Python code to Compile and fit both models. Once again, don't forget the hyperparameters.

(d) Write Python code to evaluate and compare the performance of the two models to determine which one is better. Additionally, explain why it is beneficial to divide the data into training, validation, and test sets.

Solution:

(a) Extend `model1` with another Conv2D and fully connected layer

```python
model1.add(Conv2D(num_convolutional_filers, (3, 3),
    activation="relu"))
model1.add(Flatten())
model1.add(Dense(64, activation="relu"))
model1.add(Dense(output_dimension, activation="sigmoid"))
```

(b) Extend `model2` with BatchNormalization and fully connected layer

```
model2.add(BatchNormalization()) # normalize activations
model2.add(Conv2D(num_convolutional_filers, (3, 3),
    activation="relu"))
model2.add(Flatten())
model2.add(Dense(64, activation="relu"))
model2.add(BatchNormalization()) # optional extra BN
model2.add(Dense(output_dimension, activation="sigmoid"))
```

(c) Compile and fit both models

We use `binary_crossentropy` as loss and the Adam optimizer.

```
for model in [model1, model2]:
    model.compile(
        optimizer="adam",
        loss="binary_crossentropy"
    )

    model.fit(
        X_train, y_train,
        epochs=10, # arbitrary
        batch_size=32, # arbitrary
        validation_data=(X_val, y_val),
    )
```

(d) Evaluate, compare performance, and explain the data split

```
# Evaluate on the test set
loss1, acc1 = model1.evaluate(X_test, y_test)
loss2, acc2 = model2.evaluate(X_test, y_test)

print("Model 1 - Test loss:", loss1, "Test accuracy:", acc1)
print("Model 2 - Test loss:", loss2, "Test accuracy:", acc2)

if acc1 > acc2:
    print("Model 1 performs better on the test set.")
elif acc2 > acc1:
    print("Model 2 performs better on the test set.")
else:
    print("Both models perform equally well.")
```

Why split data into training, validation, and test sets?

- **Training set**: used to learn the parameters (weights) of the model.
- **Validation set**: used to tune hyperparameters (e.g. learning rate, number of layers, number of filters, regularization strength) and to make model selection decisions (e.g. which architecture to choose, when to stop training via early stopping). It helps detect overfitting to the training data.
- **Test set**: used only at the very end to estimate the true generalization performance of the final chosen model. It must not influence training or model selection, otherwise the performance estimate becomes optimistically biased.

Question 3

(a) What benefits does using convolutional layers offer over fully connected layers in image classification tasks?

(b) Why is padding necessary when applying a filter with a stride to an input image?

(c) Given an input volume of 64x64x3 and applying 7 filters of size 6x6 with a stride of 2 and padding of 2, what will be the dimensions of the output volume?

Solution:

(a) Benefits of convolutional layers

- **Local connectivity**: convolutional filters look at small local regions (receptive fields) of the image, which matches the spatial structure of images.
- **Parameter sharing**: the same filter (set of weights) is applied across the entire image, drastically reducing the number of parameters compared to a fully connected layer. This reduces memory and overfitting.
- **Translation invariance**: features detected at one location can also be detected at another location, making the network more robust to shifts in the input.
- **Computational efficiency**: fewer parameters and shared operations lead to more efficient training and inference than fully connected layers on raw image pixels.

(b) Why padding is necessary

- Without padding, the spatial size of the feature maps shrinks after each convolution. With stride > 1, this shrinkage is even faster.
- Padding allows us to:
  - Control the output size (e.g. "same" padding keeps the width and height similar).
  - Ensure that pixels at the borders of the image are not underrepresented (otherwise border pixels are used in fewer convolution operations).
  - Prevent the feature map from becoming too small after several layers.

(c) Output volume dimensions

We use the formula for the output spatial size of a convolution:

$$\text{Output size} = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

where:

- W = input size (here 64),
- F = filter size (here 6),
- P = padding (here 2),
- S = stride (here 2).

So for width (and similarly height):

$$\text{Output size} = \left\lfloor \frac{64 - 6 + 2 \cdot 2}{2} \right\rfloor + 1 = \left\lfloor \frac{64 - 6 + 4}{2} \right\rfloor + 1 = \left\lfloor \frac{62}{2} \right\rfloor + 1 = 31 + 1 = 32.$$

The number of filters is 7, so the depth of the output volume is 7.

$$\Rightarrow \text{Output volume: } 32 \times 32 \times 7.$$

Question 4

a) How is the K-Nearest Neighbor algorithm trained?

b) How is the K-Nearest Neighbor algorithm tested?

c) How would you identify the right number of K-neighbors for a given task?

Solution:

(a) How K-Nearest Neighbor is trained

K-Nearest Neighbor (KNN) is a *lazy* learning algorithm:

- There is essentially no explicit training phase.
- "Training" consists of **storing the training data** (feature vectors and their labels).

(b) How K-Nearest Neighbor is tested

To classify a new test sample:

- Compute the distance (e.g. Euclidean distance) from the test point to **all** training points.
- Find the **K** closest (nearest) neighbors according to this distance.
- For classification: perform a **majority vote** over the labels of these K neighbors (optionally weighted by distance).
- For regression: take the **average** (or weighted average) of the target values of the K neighbors.

(c) How to choose the right value of K

- Use a **validation set** or **cross-validation**:
  - Split the training data into training and validation sets.
  - For a range of values (e.g. $K = 1, 2, 3, 4, \ldots, 20$), train KNN (store data) and measure performance on the validation set.
  - Choose the K that achieves the best validation performance.
- There is a trade-off:
  - Small K (e.g. $K = 1$) gives low bias but high variance (very sensitive to noise).
  - Large K gives smoother decision boundaries (higher bias, lower variance) but may underfit.